

The Foundation - System Mission, Ubuntu, Terminal, and GitHub

DR. TNOURJI, ABDELLAH

Research Scientist

The UM6P Vanguard Center

Autonomous Drone Engineering Internship

Mohammed VI Polytechnic University, UM6P , September 8, 2026



University
Mohammed VI
Polytechnic

THE UM6P
VANGUARD
CENTER

#Empowering Minds.

1 Understanding Ubuntu and the Linux Environment

1 Understanding Ubuntu and the Linux Environment

What Is an Operating System?

The coordinator between the computer, applications, and user

User and applications $\longleftrightarrow$ **Operating system** $\longleftrightarrow$ **Hardware**

What Is an Operating System?

The coordinator between the computer, applications, and user

User and applications $\longleftrightarrow$ **Operating system** $\longleftrightarrow$ **Hardware**

Intuitive Analogy

Think of the operating system as the manager of a laboratory. It assigns resources, organizes equipment, and controls access.

What It Manages

- ▶ files and storage;
- ▶ memory and running programs;
- ▶ users and permissions;
- ▶ devices, networks, and services.

Project Connection

When Gazebo uses the GPU, PX4 runs as a process, or ROS 2 reads a camera, the operating system coordinates the required resources.

Who Manages the Computer?

Programs need controlled access to physical resources

Active Programs (The Demand)

During the drone project, multiple heavy programs run simultaneously:

- ▶ **Gazebo:** simulates physics;
- ▶ **PX4:** controls flight logic;
- ▶ **ROS 2:** exchanges messages;
- ▶ **YOLOv8:** detects objects.

Who Manages the Computer?

Programs need controlled access to physical resources

Active Programs (The Demand)

During the drone project, multiple heavy programs run simultaneously:

- ▶ **Gazebo**: simulates physics;
- ▶ **PX4**: controls flight logic;
- ▶ **ROS 2**: exchanges messages;
- ▶ **YOLOv8**: detects objects.

Shared Resources (The Supply)

All these programs must compete for the same physical hardware:

- ▶ **Processor** (CPU time);
- ▶ **Memory** (RAM allocation);
- ▶ **Files** (Storage access);
- ▶ **Devices** (GPU and Network).

Who Manages the Computer?

Programs need controlled access to physical resources

Active Programs (The Demand)

During the drone project, multiple heavy programs run simultaneously:

- ▶ **Gazebo**: simulates physics;
- ▶ **PX4**: controls flight logic;
- ▶ **ROS 2**: exchanges messages;
- ▶ **YOLOv8**: detects objects.

Shared Resources (The Supply)

All these programs must compete for the same physical hardware:

- ▶ **Processor** (CPU time);
- ▶ **Memory** (RAM allocation);
- ▶ **Files** (Storage access);
- ▶ **Devices** (GPU and Network).

The Fundamental Problem & Key Intuition

These programs cannot safely share resources independently. If Gazebo and YOLOv8 both demand 100% of the GPU at the same time, the system crashes.

Solution: Between the software and the physical computer, we need a trusted, central traffic controller. This low-level manager is the **kernel**.

The Kernel

The low-level manager between software and hardware

Applications and Robotics Programs

Gazebo PX4 ROS 2 Python YOLOv8



Linux Kernel

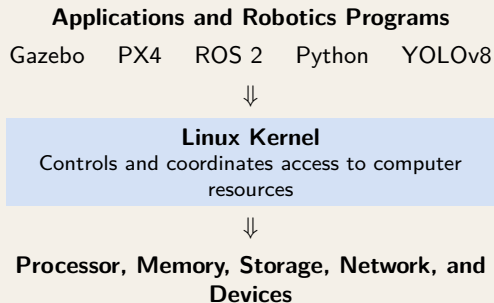
Controls and coordinates access to computer resources



**Processor, Memory, Storage, Network, and
Devices**

The Kernel

The low-level manager between software and hardware



What Does the Kernel Manage?

- ▶ **Processes:** which programs are running and when they use the processor;
- ▶ **Memory:** which memory regions belong to each running program;
- ▶ **Files and storage:** how data is read, written, and protected;
- ▶ **Devices:** how software communicates with hardware through drivers;

Linux Is More Than a Kernel

From the technical core to a complete operating environment

1. The Linux Kernel

The low-level engine that talks directly to the hardware:

- ▶ processes;
- ▶ memory;
- ▶ files & networks;
- ▶ hardware devices.

2. User Tools & Ecosystem

Software that makes the kernel actually usable by humans:

- ▶ a shell (Bash);
- ▶ terminal commands;
- ▶ software libraries;
- ▶ a package manager (apt).

3. Linux Distribution

A complete, ready-to-use operating system assembled from both parts.

Ubuntu 22.04

Linux Is More Than a Kernel

From the technical core to a complete operating environment

1. The Linux Kernel

The low-level engine that talks directly to the hardware:

- ▶ processes;
- ▶ memory;
- ▶ files & networks;
- ▶ hardware devices.

2. User Tools & Ecosystem

Software that makes the kernel actually usable by humans:

- ▶ a shell (Bash);
- ▶ terminal commands;
- ▶ software libraries;
- ▶ a package manager (apt).

3. Linux Distribution

A complete, ready-to-use operating system assembled from both parts.

Ubuntu 22.04

The Operating System Equation

Kernel + Tools, Shell & Apps = Distribution (Ubuntu)

Linux Is More Than a Kernel

From the technical core to a complete operating environment

The Operating System Equation

Kernel + Tools, Shell & Apps = Distribution (Ubuntu)



Operating System Comparison

Why don't we just program the drone on Windows or macOS?

Ubuntu (Linux)

The Robotics Standard

- ▶ **Native:** ROS 2 and Gazebo are built primarily for this OS.
- ▶ **Open:** Complete, unfiltered access to hardware.
- ▶ **Focus:** Command-line automation.

Windows

The General Desktop

- ▶ **Familiar:** Broadest consumer support.
- ▶ **Focus:** Graphical User Interfaces (GUI).
- ▶ **Barrier:** Requires Virtual Machines or WSL to run Linux tools, adding overhead.

macOS

The Premium Workspace

- ▶ **Unix-based:** Excellent developer environment.
- ▶ **Closed:** Locked to Apple hardware.
- ▶ **Barrier:** Compiling complex robotics binaries natively is often unsupported.

Operating System Comparison

Why don't we just program the drone on Windows or macOS?

Ubuntu (Linux)

The Robotics Standard

- ▶ **Native:** ROS 2 and Gazebo are built primarily for this OS.
- ▶ **Open:** Complete, unfiltered access to hardware.
- ▶ **Focus:** Command-line automation.

Windows

The General Desktop

- ▶ **Familiar:** Broadest consumer support.
- ▶ **Focus:** Graphical User Interfaces (GUI).
- ▶ **Barrier:** Requires Virtual Machines or WSL to run Linux tools, adding overhead.

macOS

The Premium Workspace

- ▶ **Unix-based:** Excellent developer environment.
- ▶ **Closed:** Locked to Apple hardware.
- ▶ **Barrier:** Compiling complex robotics binaries natively is often unsupported.

The Engineering Decision

While Windows and macOS are excellent for daily tasks, forcing them to run advanced robotics software requires workarounds (like Virtual Machines) that drain CPU and RAM. **Ubuntu provides the native, zero-overhead environment required for the drone's digital twin.**

Why Robotics Often Uses Ubuntu

A practical meeting point for robotics software

Technical Reasons

- ▶ official ROS 2 binary packages;
- ▶ mature development and command-line tools;
- ▶ convenient package and dependency management;
- ▶ strong support for Python, C++, containers, and Git.

Team Reasons

- ▶ shared installation instructions;
- ▶ repeatable scripts and workspaces;
- ▶ extensive community documentation;
- ▶ easier comparison of errors and solutions.

Project Connection

Our Ubuntu environment hosts ROS 2, Gazebo, PX4, Python tools, computer-vision code, Git, and the internship workspace.

Why One Exact Ubuntu Version Matters

Compatibility is part of engineering, not an installation detail

Ubuntu version → available packages → compatible robotics stack

Compatibility is part of engineering, not an installation detail

Compatibility is part of engineering, not an installation detail

Ubuntu version → available packages → compatible robotics stack

Example Compatibility Pairs

- ▶ ROS 2 Humble: Ubuntu 22.04
- ▶ ROS 2 Jazzy: Ubuntu 24.04

Internship Rule

Follow the version declared in the project documentation. Do not replace it only because another release is newer.

Why This Matters

A tutorial written for another Ubuntu or ROS 2 release may contain different package names, dependencies, or setup commands.

The Filesystem: One Organized Tree

Linux organizes files from a single starting point

`/` → `/home` → `/home/student` → project workspace

The Filesystem: One Organized Tree

Linux organizes files from a single starting point

`/` → `/home` → `/home/student` → project workspace

Filesystem

The rule and structure used to organize files and directories. The top of the Linux tree is called **root** and is written as `/`.

Home Directory

Each user normally works inside a personal directory such as `/home/student`. The shortcut `~` means “my home directory.”

Project Connection

File paths tell scripts where to find worlds, models, source code, configuration files, logs, and datasets.

Users, Administrator, and Root

Permissions protect the system from accidental changes

User

A normal account for daily work, coding, simulation, and documents.

Administrator

A trusted user allowed to perform selected system-level tasks.

Root

The system's all-powerful administrative account. Root can modify almost anything.

Users, Administrator, and Root

Permissions protect the system from accidental changes

User

A normal account for daily work, coding, simulation, and documents.

Administrator

A trusted user allowed to perform selected system-level tasks.

Root

The system's all-powerful administrative account. Root can modify almost anything.

What sudo Means

sudo requests permission to run one command with administrative privileges. It is not a decoration to add when a command fails.

Safe Working Habit

Work as a normal user. Use administrative privileges only when the command is understood and the trusted documentation requires them.

Packages, Dependencies, and Repositories

How Ubuntu obtains and maintains software

Package

A prepared unit of software with metadata needed for installation and maintenance.

Dependency

Another package or library required for the software to work.

Repository

A configured software source from which packages and updates can be obtained.

Terminal, Shell, and Bash

The window, the interpreter, and the specific tool

Terminal

The application window that displays text and accepts keyboard input.

Shell

The command interpreter that reads instructions and starts programs.

Bash

A widely used shell that supports commands, variables, scripts, redirection, and automation.

Terminal, Shell, and Bash

The window, the interpreter, and the specific tool

Terminal

The application window that displays text and accepts keyboard input.

Shell

The command interpreter that reads instructions and starts programs.

Bash

A widely used shell that supports commands, variables, scripts, redirection, and automation.

Restaurant Analogy

The terminal is the counter, the shell is the person interpreting the order, and Bash is one specific language and rule set used to express that order.

Next tutorial: we will use the terminal and Bash safely, one command at a time.

Open Source Is More Than Free Access

It is a collaborative engineering model

Open-Source Software

Source code is available under a license that defines how it may be inspected, used, modified, and shared.

Community Collaboration

Developers contribute code, reviews, documentation, tests, bug reports, and technical discussions.

Professional Habit

Reusing open-source software does not remove engineering responsibility. We still verify versions, licenses, interfaces, behavior, and assumptions.

Modular Software

Complex systems are divided into components with clear responsibilities and interfaces.

Documentation

Installation steps, assumptions, versions, and known problems are recorded so that others can repeat the work.

Reproducibility: “It Works on My Computer” Is Not Enough

Another student should be able to obtain the same result

Same repository + same versions + documented steps → comparable result

Reproducibility: “It Works on My Computer” Is Not Enough

Another student should be able to obtain the same result

Same repository + same versions + documented steps → comparable result

Record

- ▶ Ubuntu and ROS 2 versions;
- ▶ required packages and dependencies;
- ▶ configuration and environment variables;
- ▶ launch and test commands.

Verify

- ▶ start from a clean terminal;
- ▶ follow the documented sequence;
- ▶ compare expected and observed outputs;
- ▶ report missing assumptions.

Error Messages Are Engineering Evidence

Troubleshooting begins with observation, not random commands

1. **Stop and read** the complete message.
2. **Locate** the first useful error, file, package, or process name.
3. **Check context**: directory, command, versions, and environment.
4. **Form one hypothesis** and test one change at a time.
5. **Document the verified fix** and how it was validated.

Error Messages Are Engineering Evidence

Troubleshooting begins with observation, not random commands

1. **Stop and read** the complete message.
2. **Locate** the first useful error, file, package, or process name.
3. **Check context**: directory, command, versions, and environment.
4. **Form one hypothesis** and test one change at a time.
5. **Document the verified fix** and how it was validated.

Beginner-Friendly Question

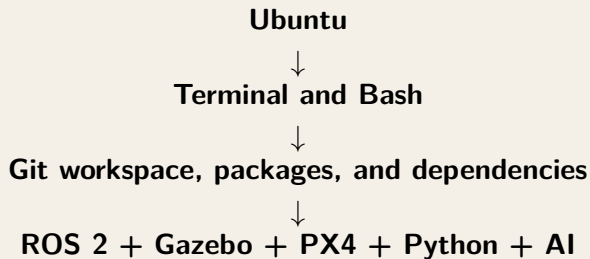
Instead of asking “Why does nothing work?”, ask: “Which component failed, what evidence identifies it, and what changed since the last working state?”

Avoid

Blindly adding sudo, repeatedly reinstalling everything, or copying commands written for another software version.

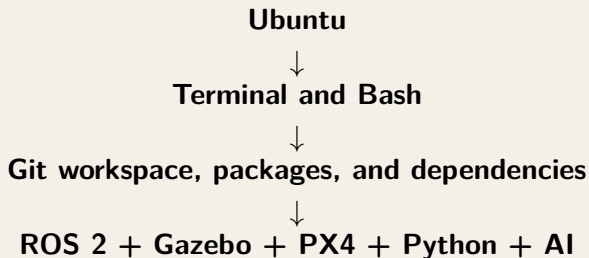
Ubuntu Inside the Drone Project

The operating environment supports every software layer



Ubuntu Inside the Drone Project

The operating environment supports every software layer



System-Level Mental Model

Ubuntu does not perform the inspection mission by itself. It provides the common environment in which the simulation, communication, flight-control, programming, and perception tools run.

Simulation Boundary

A working Ubuntu simulation is a valuable development result, but it does not by itself

What You Should Now Be Able to Explain

Conceptual checkpoint before using the command line

- ▶ An **operating system** coordinates applications, users, and hardware.
- ▶ **Ubuntu** is a Linux distribution; **Linux** provides the kernel foundation.
- ▶ Ubuntu is selected because it offers a supported and reproducible robotics environment.
- ▶ A **filesystem** organizes files; users and permissions control access.
- ▶ Packages come from repositories and may require dependencies.
- ▶ Running programs are processes that receive settings from their environment.
- ▶ Open-source robotics depends on collaboration, modularity, documentation, reproducibility, and disciplined troubleshooting.

What You Should Now Be Able to Explain

Conceptual checkpoint before using the command line

- ▶ An **operating system** coordinates applications, users, and hardware.
- ▶ **Ubuntu** is a Linux distribution; **Linux** provides the kernel foundation.
- ▶ Ubuntu is selected because it offers a supported and reproducible robotics environment.
- ▶ A **filesystem** organizes files; users and permissions control access.
- ▶ Packages come from repositories and may require dependencies.
- ▶ Running programs are processes that receive settings from their environment.
- ▶ Open-source robotics depends on collaboration, modularity, documentation, reproducibility, and disciplined troubleshooting.

Next: Terminal and Bash Foundations

Learning Roadmap

Each foundation unlocks one part of the final GitHub project

1. **Ubuntu** provides the common engineering environment.
2. **Terminal and Bash** make the workflow visible and repeatable.
3. **Git and GitHub** preserve, document, and share the project.
4. **Python** implements robotics logic and data processing.
5. **ROS 2** connects the software components.
6. **Gazebo and PX4** provide the simulated world and flight behavior.
7. **AI integration** converts camera images into useful observations.
8. **System testing and documentation** turn components into a reproducible demonstration.

Destination

Clone the GitHub repository, configure the documented environment, launch the complete system, observe the drone inspection pipeline, diagnose problems, and deliver verified technical evidence.

References and Further Reading I

Ubuntu and Linux foundations



Ubuntu, *About the Ubuntu Project*.

<https://ubuntu.com/about>



Canonical, *Official Ubuntu Documentation*.

<https://help.ubuntu.com/>



Canonical, *Ubuntu Desktop Documentation*.

<https://documentation.ubuntu.com/desktop/>



The Linux Kernel Organization, *Linux Kernel Documentation*.

<https://docs.kernel.org/>



Linux Foundation, *Filesystem Hierarchy Standard*.

<https://refspecs.linuxfoundation.org/fhs.shtml>

References and Further Reading II

Packages, permissions, and robotics compatibility



Ubuntu, *Ubuntu Packages Search*.
<https://packages.ubuntu.com/>



Debian Project, *APT User's Guide*.
<https://www.debian.org/doc/manuals/apt-guide/>



Ubuntu Community Help Wiki, *RootSudo*.
<https://help.ubuntu.com/community/RootSudo>



Open Robotics, *ROS 2 Humble Installation on Ubuntu 22.04*.
<https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html>



Open Robotics, *ROS 2 Jazzy Installation on Ubuntu 24.04*.
<https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html>